

Local RAG Agent

Team Tutorial Guide

A beginner-friendly guide to RAG (Retrieval-Augmented Generation)
and how to use this offline Ollama chatbot with your documents.

Source: RAG_Agent_Team_Tutorial.md

1. Introduction

This project is a local chatbot that answers questions about your documents. It runs entirely on your computer:

- No document data is sent to the cloud
- Ollama runs the AI models locally
- ChromaDB stores searchable vector indexes on disk

You can drag and drop PDF, Word, text, and code files into a web page, choose embedding and chat models from dropdown menus (populated from your installed Ollama models), build a knowledge base, and ask questions. Answers use inline references like [1] only in the text; source sentences appear in a Reference quotes section below.

2. What is RAG?

2.1 The problem with plain chatbots

A normal Large Language Model (LLM) only knows what it was trained on. If you ask:

> "What is our company's refund policy?"

...it will guess or give generic advice unless that policy was in its training data. It cannot read your internal PDFs by default.

2.2 What RAG adds

RAG = Retrieval-Augmented Generation

Step | Name | What happens

- 1 | Retrieval | Find the most relevant pieces of **your** documents
- 2 | Augmentation | Put those pieces into the prompt sent to the LLM
- 3 | Generation | The LLM writes an answer **grounded in those pieces**

Think of RAG as: "Open the right pages of the manual, then ask the AI to explain only what is on those pages."

2.3 Simple analogy

Role | RAG equivalent

Library | Your uploaded files (indexed on disk)

Librarian | Vector search (finds similar text to your question)

Expert reader | Ollama chat model (writes the answer)

Footnotes | [1], [2] citations in the answer

2.4 Embeddings (intuition)

Computers cannot directly compare "refund policy" with a paragraph of text by meaning. Instead, text is converted into a list of numbers called an embedding (a vector). Similar meanings produce similar vectors.

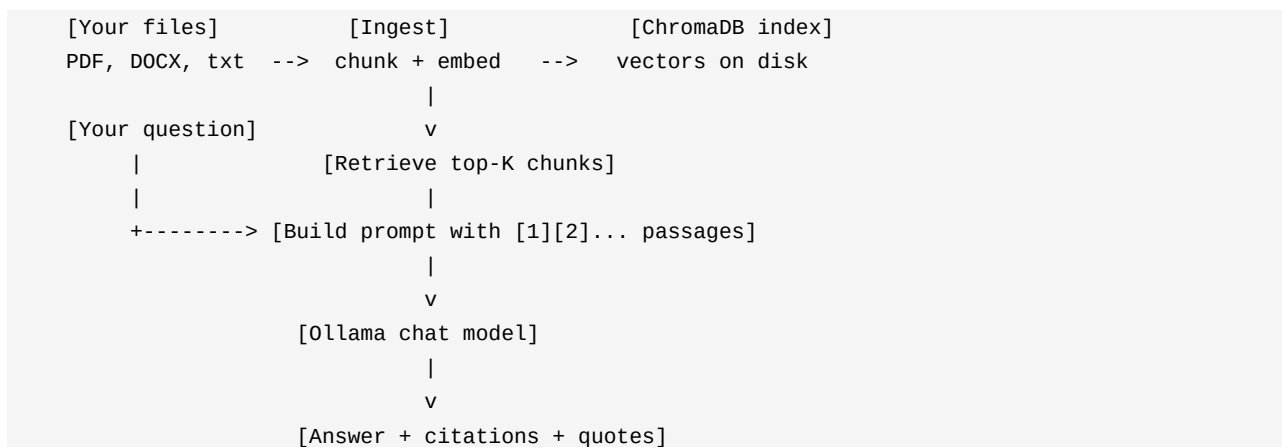
When you ask a question:

- The question is embedded
- The system finds document chunks with the closest vectors
- Those chunks are sent to the LLM

By default this project uses Ollama model nomic-embed-text for embeddings and llama3.2 for chat - but in the web UI you can pick any installed model from dropdown lists.

3. How This Project Works

3.1 End-to-end flow



3.2 Two types of Ollama models

RAG needs two different models - they are not interchangeable:

Type | Default example | Role

Embedding model | nomic-embed-text | Turns text into vectors for search

Chat model | llama3.2 | Reads retrieved text and writes answers

Install at least one of each:

```
ollama pull nomic-embed-text
ollama pull llama3.2
```

In the web UI, dropdown menus list every model you have pulled (ollama pull ...). The app detects model type using Ollama's capabilities field (e.g. embedding vs completion).

Rule: Use the same embedding model for indexing and searching. If you change the embedding model after building a knowledge base, you must Replace entire knowledge base and re-ingest all files.

3.3 Chunking

Documents are split into chunks (~800 characters) with overlap (~120 characters) so sentences at boundaries are not lost. Each chunk remembers:

- Source file path
- Start and end line numbers (for citations)

3.4 Offline guarantee

- Ollama runs at <http://localhost:11434>
- ChromaDB stores data in the index/ folder
- Uploads are copied to uploads/ (optional persistence)

Nothing in this pipeline requires an OpenAI or cloud API key.

4. Key Concepts Glossary

Term | Definition

LLM | Large Language Model - predicts text (e.g. Llama via Ollama)

Embedding | Numeric representation of text meaning

Vector database | Stores embeddings and finds nearest neighbors (ChromaDB)

Chunk | A small segment of a document used for retrieval

Top-K | How many chunks to retrieve per question (default: 6)

Ingest / Index | Process of reading files and storing chunks in ChromaDB

Knowledge base | All indexed chunks taken together

Citation | Reference like [1] pointing to a retrieved passage

Verbatim quote | Exact copy of source text inside quotation marks

Hallucination | When an LLM invents facts not in the sources - RAG reduces this

Embedding model | Model that only creates vectors (not chat replies)

Chat model | Model that generates natural-language answers

5. Installation & Setup

5.1 Prerequisites

- Windows 10/11 (or macOS/Linux)
- Python 3.10+ - <https://www.python.org/downloads/>
- Ollama - <https://ollama.com/download>

5.2 Install Ollama models

Open a terminal (PowerShell or Command Prompt):

```
ollama pull nomic-embed-text
ollama pull llama3.2
```

Verify Ollama is running (system tray icon or ollama list).

5.3 Install the RAG Agent

```
cd path\to\RAG-Agent
python -m venv .env
.venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
```

5.4 Start the chatbot (easiest)

Option A - Double-click: run_chatbot.bat

Option B - Terminal:

```
python -m src serve
```

Open <http://127.0.0.1:7860> in your browser.

6. Using the Web Chatbot

6.1 Step-by-step

- Start run_chatbot.bat or python -m src serve
- Open <http://127.0.0.1:7860>
- Check sidebar - should say "Ollama: connected", model counts, and chunk count
- Select models (top of sidebar):
- Embedding model - used when you click Add to knowledge base
- Chat model - used when you send a chat message

- Drag files into "Drop files here" (PDF, DOCX, TXT, MD, code, etc.)
- Click Add to knowledge base
- Wait for confirmation (e.g. "Indexed 3 file(s), 42 chunk(s)")
- Type a question in the chat box and press Send

6.2 Model selection dropdowns

The UI loads models from ollama list and classifies them automatically:

Dropdown | Lists models with... | Used when...

Embedding model | embedding capability (e.g. nomic-embed-text:latest) | Indexing / ingesting documents

Chat model | completion, chat, or tools (e.g. llama3.2:latest) | Answering your questions

Button / action | Effect

Refresh model list | Re-scans Ollama after you run ollama pull <model>

Page load | Dropdowns auto-fill on startup

The status panel shows:

- How many embedding and chat models are installed
- Which models are currently selected
- Which embedding model built the existing index (Indexed with: ...)
- A warning if the selected embedding model differs from the indexed one

After pulling a new model:

```
ollama pull mistral
ollama pull mxbai-embed-large
```

Click Refresh model list in the UI - the new names appear in the dropdowns.

6.3 Sidebar controls (documents & index)

Control | Effect

Replace entire knowledge base | Deletes old index before adding new files (required when changing embedding model)

Clear knowledge base | Empties the vector index

Also delete saved uploads | Removes files from uploads/ folder too

Clear chat | Clears conversation only (not the index)

6.4 Supported file types

- PDF (.pdf) - text extraction per page
 - Word (.docx) - paragraphs and tables
 - Text & code - .txt, .md, .py, .json, .csv, .html, and many more
- Not supported: scanned PDFs (images only), legacy .doc files, photos.

6.5 Example questions

- "What is the refund policy?"
- "Summarize the authentication section."
- "What are the support hours?"

7. Using the Command Line

For automation or scripting:

Command | Purpose

python -m src serve | Launch web UI

```
python -m src ingest .documents | Index a folder
python -m src ingest .documents --reset | Rebuild index from scratch
python -m src ask "your question" | Single question (uses chat model from .env)
python -m src ask "..." --model mistral | Single question with a specific chat model
python -m src chat | Terminal chat loop
python -m src status | Show index size and models
```

> Note: Model dropdowns are a web UI feature. The CLI uses OLLAMA_CHAT_MODEL and OLLAMA_EMBED_MODEL from .env unless you pass --model for ask/chat.

8. How Answers Include Citations

8.1 Retrieval labels

Each retrieved chunk is numbered [1], [2], ... with a location tag:

```
[1] policy.txt:L12-18
-----
(verbatim chunk text here)
```

8.2 Answer rules (enforced by prompts)

The chat model is instructed to:

- Cite claims with [N] immediately after the fact
- Include exact quotes in double quotes from the passage
- Not invent information outside the passages

8.3 Source appendix

The answer body contains only [N] markers - no quoted sentences inline. After the answer, a Reference quotes section lists the relevant source sentence(s) for each [N].

8.4 Quote validation

The code checks whether quoted strings actually appear in the cited passage and shows warnings in the UI if not.

9. Project Structure

```
RAG-Agent/
??? config.py           # Settings (default models, paths, chunk size)
??? run_chatbot.bat      # Windows launcher
??? generate_tutorial_pdf.bat
??? requirements.txt     # Python dependencies
??? .env.example         # Environment variable template
??? documents/           # Example documents (optional)
??? uploads/             # Saved uploads from web UI
??? index/              # ChromaDB vector store (auto-created)
??? docs/
?   ??? RAG_Agent_Team_Tutorial.md # This guide (source)
?   ??? RAG_Agent_Team_Tutorial.pdf # Shareable PDF (generated)
??? scripts/
?   ??? generate_tutorial_pdf.py
??? src/
    ??? documents.py     # Read & chunk files (PDF, DOCX, text)
    ??? ollama_models.py # List & classify installed Ollama models
```

```
??? store.py          # ChromaDB + Ollama embeddings
??? retrieve.py       # Semantic search
??? agent.py          # LLM prompts & citations
??? ingest_service.py # Upload & index orchestration
??? app.py            # Gradio web UI (dropdowns, chat, upload)
??? cli.py            # Terminal commands
```

10. Code Walkthrough

This section explains what each file does and important functions.

10.1 config.py - Central settings

Loads environment variables from .env using python-dotenv.

Setting | Default | Meaning

OLLAMA_HOST | http://localhost:11434 | Where Ollama API listens

OLLAMA_EMBED_MODEL | nomic-embed-text | Embedding model name

OLLAMA_CHAT_MODEL | llama3.2 | Chat model name

CHROMA_PATH | ./index | Vector database folder

UPLOADS_DIR | ./uploads | Stored upload copies

CHUNK_SIZE | 800 | Target chunk size (characters)

CHUNK_OVERLAP | 120 | Overlap between chunks

TOP_K | 6 | Chunks retrieved per query

APP_HOST | 127.0.0.1 | Web UI bind address

APP_PORT | 7860 | Web UI port

TEXT_EXTENSIONS and OFFICE_EXTENSIONS define which file types the ingest pipeline accepts.

Web UI vs .env: Dropdown selections in the browser override defaults at runtime. .env values are used as fallbacks when the UI loads and for CLI commands.

10.2 src/documents.py - Reading and chunking files

Purpose: Turn files on disk into TextChunk objects ready for indexing.

TextChunk (dataclass)

Stores one chunk's:

- text - the actual content
- source_path - full path to the original file
- start_line / end_line - for citations like file.pdf:L10-25
- chunk_index - order within the file
- chunk_id - unique hash ID for ChromaDB

read_pdf_file(path)

Uses pypdf PdfReader to extract text page by page. Each page is prefixed with --- Page N --- so page numbers appear in citations.

read_docx_file(path)

Uses python-docx to read paragraphs and table rows (cells joined with |).

read_text_file(path)

Reads plain files with UTF-8 (fallback: Latin-1, CP1252). Skips binary files (null bytes in header).

read_document_file(path)

Router: .pdf -> PDF reader, .docx -> DOCX reader, else -> text reader.

chunk_text(text, source_path, chunk_size, chunk_overlap)

Splits text line by line into overlapping chunks without breaking mid-line when possible. Overlap ensures

context at chunk boundaries is preserved for retrieval.

load_chunks_from_path / load_chunks_from_root

High-level: read file(s) -> return list of TextChunk.

10.3 src/store.py - Vector database

Purpose: Save chunks in ChromaDB with Ollama-generated embeddings.

_embedding_function(model_name)

Returns OllamaEmbeddingFunction for the given model name - Chroma calls Ollama when adding or querying documents.

get_collection(reset=False, embed_model=None)

Opens (or creates) persistent collection rag_documents.

- If embed_model is passed (from the UI), that model is used for indexing.
- If not passed, the model stored in collection metadata is reused (so search matches the index).
- If you switch embedding models on a non-empty index without reset, a ValueError explains you must re-ingest.

Collection metadata stores embed_model so the app knows which model built the index.

Uses cosine similarity (hnsw:space: cosine).

upsert_chunks(collection, chunks)

Writes chunks in batches of 64:

- ids - unique chunk_id
- documents - chunk text
- metadatas - file path, line numbers

upsert means update if ID exists, insert if new.

10.4 src/retrieve.py - Semantic search

Purpose: Given a user question, find the most relevant chunks.

RetrievedPassage

Wraps one search result with ref_id (1, 2, 3...), text, location, and distance (lower = more similar).

retrieve(query, top_k)

- Embeds the query via Ollama (through Chroma)
- Queries Chroma for nearest top_k chunks
- Returns numbered passages for the agent

format_for_prompt()

Formats a passage for the LLM:

```
[1] policy.txt:L5-12
-----
(chunk text)
```

10.5 src/agent.py - LLM answering with citations

Purpose: Orchestrate retrieval + Ollama chat + citation formatting.

SYSTEM_PROMPT

Strict instructions: cite with [N], use verbatim quotes, do not hallucinate.

ask(query, model=None, top_k=None, include_sources=True)

Main pipeline:

- retrieve(query) - get passages
- build_context(passages) - format for prompt

- ollama.chat(model=...) - uses UI-selected chat model or OLLAMA_CHAT_MODEL default
- validate_quotes_in_answer - check quotes match sources
- format_source_appendix - append full source text for cited [N]

Returns dict with answer, passages, warnings, model.

format_source_appendix

Parses [1], [2] from the answer and appends markdown code blocks with full chunk text.

validate_quotes_in_answer

Regex finds patterns like [1] ... "quoted text" and verifies the quote exists in that passage.

10.6 src/ingest_service.py - Indexing orchestration

Purpose: Shared logic for CLI and web UI uploads.

ingest_paths(paths, reset, embed_model=None)

For each file: load_chunks_from_path -> upsert_chunks. Passes embed_model to get_collection. Returns IngestResult statistics.

persist_upload(temp_path)

Copies Gradio's temporary upload file into uploads/ with a unique name if duplicate.

ingest_uploads(file_paths, reset, embed_model=None)

Save uploads -> ingest paths with the selected embedding model.

clear_knowledge_base(delete_uploads)

Resets Chroma collection; optionally deletes uploads/ folder.

10.7 src/ollama_models.py - Installed model discovery

Purpose: Populate embedding and chat dropdowns from Ollama.

list_models(refresh=False)

- Calls ollama.list() for all installed model names
- For each model, calls ollama.show(name) and reads capabilities
- Classifies:
 - Embedding - capability includes embedding
 - Chat - capability includes completion, chat, or tools
- Caches results for 30 seconds (use refresh=True when user clicks Refresh)

default_embed_choice / default_chat_choice

Pick the model matching .env defaults, or the first available.

Name-based fallback

If capabilities are missing, names containing embed are treated as embedding models.

10.8 src/app.py - Web chatbot (Gradio)

Purpose: Browser UI for non-technical users.

build_ui()

Creates Gradio layout:

- Left column: model dropdowns, refresh button, file upload, ingest, clear KB, status
- Right column: chatbot, message box, send/clear

refresh_model_dropdowns()

Re-loads model lists from Ollama and updates both dropdowns via gr.update().

handle_ingest(files, reset_index, embed_model)

Calls ingest_uploads(..., embed_model=embed_model), shows summary including which embed model was used.

handle_chat(message, history, chat_model, embed_model)

Calls ask(message, model=chat_model), appends messages in Gradio 6 format:

```
{"role": "user", "content": "..."}  
{"role": "assistant", "content": "..."}  

```

sidebar_status(embed_model, chat_model)

Shows Ollama connection, model counts, selections, index size, and embed-model mismatch warnings.

launch()

Starts server at 127.0.0.1:7860.

10.9 src/cli.py - Terminal interface

Purpose: Command-line access to the same backend.

Uses argparse subcommands: ingest, ask, chat, status, serve.

Each command calls the same modules as the web UI (ingest_paths, ask, launch).

Uses rich library for colored terminal output and markdown rendering.

11. Configuration Reference

Edit .env in the project root (copy from .env.example):

```
OLLAMA_HOST=http://localhost:11434  
OLLAMA_EMBED_MODEL=nomic-embed-text  
OLLAMA_CHAT_MODEL=llama3.2  
CHROMA_PATH=./index  
UPLOADS_DIR=./uploads  
CHUNK_SIZE=800  
CHUNK_OVERLAP=120  
TOP_K=6  
APP_HOST=127.0.0.1  
APP_PORT=7860
```

Important: If you change the embedding model after indexing, you must re-ingest all documents. In the web UI, enable Replace entire knowledge base; in the CLI, use --reset.

Defaults vs UI: OLLAMA_EMBED_MODEL and OLLAMA_CHAT_MODEL in .env set initial dropdown values and CLI defaults. The web UI lets users switch models per session without editing .env.

12. Troubleshooting

Problem | Solution

"Ollama: not reachable" | Start Ollama app; check http://localhost:11434

Empty model dropdowns | Run ollama pull nomic-embed-text and ollama pull llama3.2; click Refresh model list

New model not in dropdown | Run ollama pull <model> then Refresh model list

"Knowledge base was built with embedding model X..." | Enable Replace entire knowledge base and re-ingest, or switch embed dropdown back to X

"No documents indexed" | Upload files and click Add to knowledge base

PDF returns no text | PDF may be scanned images - needs OCR first

DOCX not working | Save as .docx (not old .doc)

Chat UI error on send | Restart server; chat uses Gradio 6 message format (role / content)

Slow first question | Ollama loads model into RAM - first call is slower

Wrong answers | Add more relevant files; try a larger chat model; ask focused questions

Port 7860 in use | Change APP_PORT in .env

13. Best Practices for Your Team

- Organize documents - Put related files together; use clear filenames.
- Pick models deliberately - Use a strong embed model for search quality; use a capable chat model for answer quality. They can differ.
- Do not mix embedding models - Re-index with Replace entire knowledge base when changing the embed dropdown.
- Re-index after updates - If a policy PDF changes, upload again (or use "Replace entire knowledge base").
- Ask specific questions - "What is the warranty period?" works better than "Tell me everything."
- Verify citations - Read the Reference quotes section for compliance-sensitive topics.
- Keep data local - Do not use --share on Gradio unless you intend to expose the UI publicly.
- One project folder per team - Separate indexes for HR vs Engineering if needed (different CHROMA_PATH in .env).

Quick Reference Card

```
START:      run_chatbot.bat  OR  python -m src serve
URL:        http://127.0.0.1:7860
MODELS:     Select Embedding + Chat dropdowns (Refresh after ollama pull)
UPLOAD:     Drag files -> Add to knowledge base
ASK:        Type question in chat
PULL:       ollama pull nomic-embed-text && ollama pull llama3.2
RESET:      Replace entire knowledge base (when changing embed model)
PDF HELP:   generate_tutorial_pdf.bat -> docs/RAG_Agent_Team_Tutorial.pdf
```

End of tutorial. For questions about this codebase, contact your team administrator or refer to README.md in the project root.